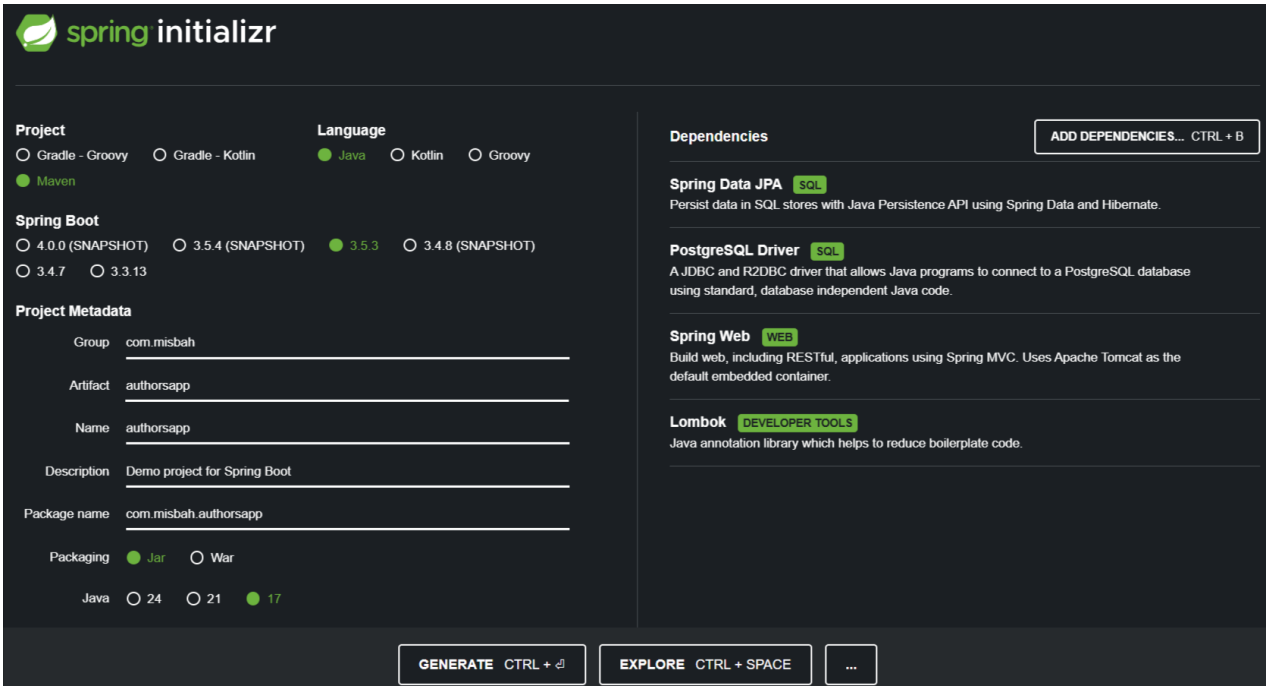
SPRINGBOOT REST API ENDPOINT AND CONNECTING TO PostgreSQL via DOCKER

Go to start.spring.io and select the following dependencies, package it ,extract and open in IntelliJ IDEA




```
src/
├── main/
│   ├── java/
│   │   └── com/misbah/
│   │       ├── model/
│   │       │   └── Author.java        (Entity)
│   │       ├── repository/
│   │       │   └── AuthorRepository.java (Repository)
│   │       ├── controller/
│   │       │   └── AuthorController.java (API)
│   │       └── AuthorsappApplication.java  (Main class)
│   └── resources/
│       └── application.properties
```

write in application.properties

```
spring.application.name=authorsapp
spring.datasource.url=jdbc:postgresql://localhost:5432/authors
db
spring.datasource.username=postgres
spring.datasource.password=pass123

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
PostgreSQLDialect

#spring.sql.init.mode=always
```

Steps In DOCKER -> OPEN DOCKER DESKTOP : confirm its running in Tray

open Terminal in Docker and write

docker run --name pgdb `

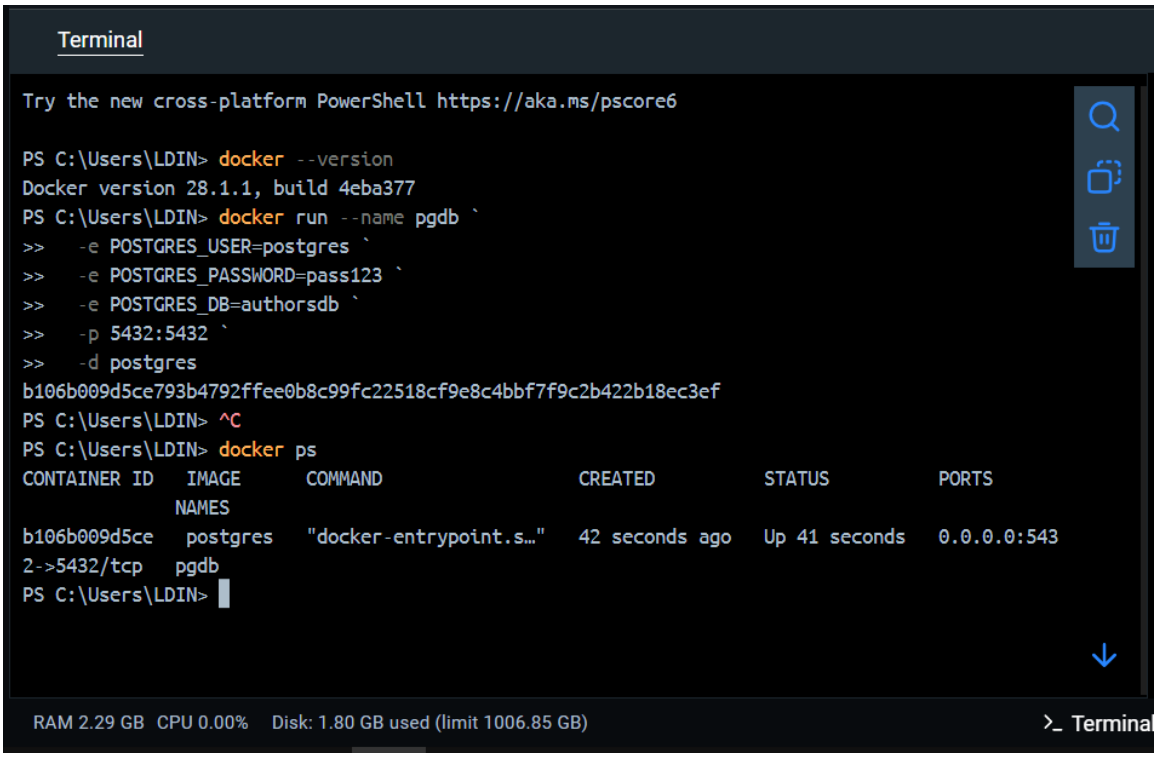
>> -e POSTGRES_USER=postgres `

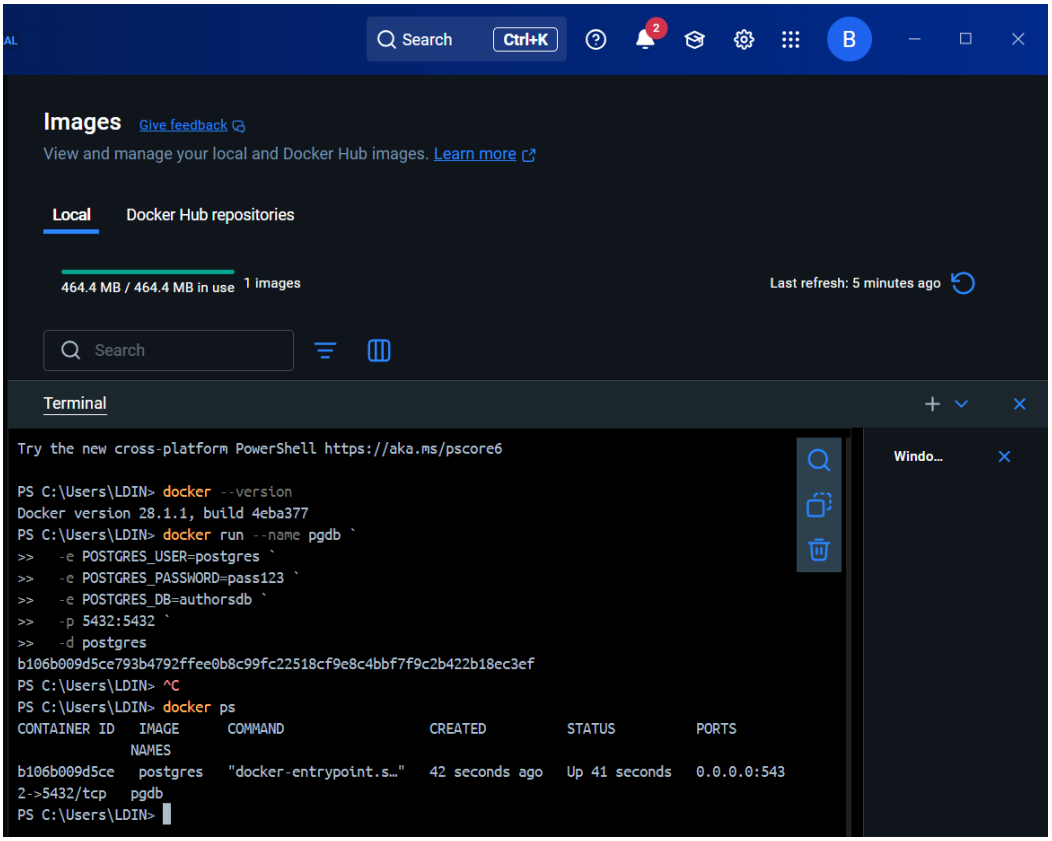
>> -e POSTGRES_DB=authorsdb `

>> -p 5432:5432 `

>> -d postgres

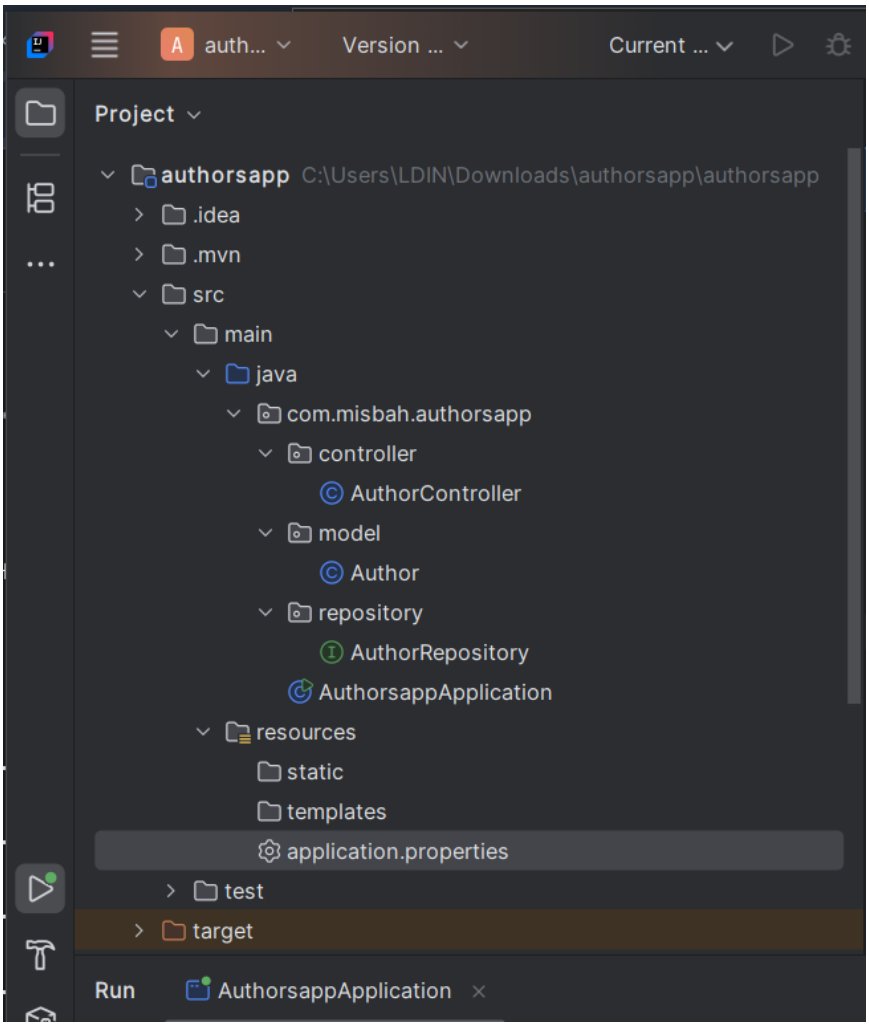
OUR application.properties and docker image command or docker-compose.yml SHOULD ALWAYS MATCH !





docker ps : will give info about all running containers

Create repository structure like so



We write the JPA Entity first in model/Author.java

```
package com.misbah.authorsapp.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name="authors")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Author{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false,length=100)
    private String name;
    private Integer age;

}
```

Create the Spring Data Repository as new INTERFACE AuthorRepository in repository/AuthorRepository

```
package com.misbah.authorsapp.repository;

import com.misbah.authorsapp.model.Author;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AuthorRepository extends JpaRepository<Author,Long>
{

}
```

Build a Simple REST Controller

Inside Controller package we create a Java Class AuthorController

```
package com.misbah.authorsapp.controller;


import com.misbah.authorsapp.model.Author;
import com.misbah.authorsapp.repository.AuthorRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/authors")
```

```java
public class AuthorController
{
    @Autowired
    private AuthorRepository authorRepository;

    @PostMapping
    public Author createAuthor(@RequestBody Author author)
    {
        return authorRepository.save(author);
    }
    @GetMapping
    public List<Author> getAllAuthors()
    {
        return authorRepository.findAll();
    }
}
```

Now run the Spring Boot App on AuthorsappApplication.java and in the terminal we see our table is created and we didn't need to write SQL code for it

```
2025-06-30T15:48:26.918+05:30  INFO 7364 --- [authorsapp] [  restartedMain] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start complete
2025-06-30T15:48:26.988+05:30  WARN 7364 --- [authorsapp] [  restartedMain] org.hibernate.orm.deprecation            : HHH90000025: PostgreSQLDialec
2025-06-30T15:48:27.020+05:30  INFO 7364 --- [authorsapp] [  restartedMain] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
    Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
    Database driver: undefined/unknown
    Database version: 17.5
    Autocommit mode: undefined/unknown
    Isolation level: undefined/unknown
    Minimum pool size: undefined/unknown
    Maximum pool size: undefined/unknown
2025-06-30T15:48:28.446+05:30  INFO 7364 --- [authorsapp] [  restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform av
Hibernate: create table authors (id bigint generated by default as identity, age integer, name varchar(100) not null, primary key (id))
2025-06-30T15:48:28.635+05:30  INFO 7364 --- [authorsapp] [  restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManager
2025-06-30T15:48:29.028+05:30  WARN 7364 --- [authorsapp] [  restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is en
2025-06-30T15:48:29.556+05:30  INFO 7364 --- [authorsapp] [  restartedMain] o.s.b.d.a.OptionalLiveReloadServer       : LiveReload server is running
2025-06-30T15:48:29.595+05:30  INFO 7364 --- [authorsapp] [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080 (
2025-06-30T15:48:29.610+05:30  INFO 7364 --- [authorsapp] [  restartedMain] c.m.authorsapp.AuthorsappApplication     : Started AuthorsappApplication
```

our server is created on htpp://localhost:8080

we created an endpoint htpp://localhost:8080/authors

we hit it using tool curl to send a request with data (an Author object)

The data was saved into a PostgreSQL database running inside Docker

we use curl to test our backend server by sending HTTP requests from your terminal

```
Command Prompt
Microsoft Windows [Version 10.0.19045.5965]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LDIN>curl -X POST http://localhost:8080/authors -H "Content-Type: application/json" -d "{\"name\":\"Walter White\", \"age\":52}"
{"id":1,"name":"Walter White","age":52}
C:\Users\LDIN>
```

curl -X POST http://localhost:8080/authors -H "Content-Type: application/json" -d "{\"name\":\"Walter White\", \"age\":52}"

```
curl -X POST http://localhost:8080/authors -H
"Content-Type: application/json" -d
"{\"name\":\"Walter White\", \"age\":52}"
```

curl : Is a command line tool to make network requests

-X POST : we use HTTP POST method to send data

http://localhost:8080/authors : This is the URL of your Spring Boot backend API

-H “Content-Type:application/json :Tells the server the format of data we are sending

-d “{..}” : d "{\"name\":\"Walter White\", \"age\":52}"

the actual data we are sending in JSON Body

The http://localhost:8080/authors is our Spring Boot Server running locally on our computer created by our Application

When we run our Spring Boot App

1.Spring boot starts an embedded web server – most commonly Tomcat -on port 8080

2.It reads your controller classes to know what endpoints to handle

3.We declared REST endpoint like :

```
@RestController
@RequestMapping("/authors")
public class AuthorController
{
    @Autowired
    private AuthorRepository authorRepository;

    @PostMapping
    public Author createAuthor(@RequestBody Author author)
    {
        return authorRepository.save(author);
    }
```

This means ,when we send a POST request to /authors on port 8080

with json body like that it will trigger this method and thus Author will be saved to database by that save() method

localhost:8080/authors

Pretty-print


```
[
  {
    "id": 1,
    "name": "Walter White",
    "age": 52
  },
```


ACCESSING POSTGRESQL DB AND TABLE INSIDE OUR DOCKER

1. Access the terminal inside your PostgreSQL container
   docker exec -it pgdb bash
   (docker exec -it your_container_name bash)
2. Enter the PostgreSQL interactive console (psql)
   psql -U postgres -d authorsdb
   (psql -U your_user -d your_db_name)

we can come to know our container name by **docker ps** and our db name in application.properties in spring.datasource.url and our username and passwords too

```
32->5432/tcp   pgdb                                                    PS C:\Users\LDIN> docker exec -it pgdb bash
root@b106b009d5ce:/# psql -U postgres -d authorsdb
psql (17.5 (Debian 17.5-1.pgdg120+1))
Type "help" for help.

authorsdb=# \dt
          List of relations
 Schema |  Name   | Type  |  Owner
--------+---------+-------+----------
 public | authors | table | postgres
(1 row)

authorsdb=# SELECT * FROM authors;
 id | age | name
----+-----+------
(0 rows)

authorsdb=# SELECT * FROM authors;
 id | age |     name
----+-----+--------------
  1 |  52 | Walter White
```

before curl command was run

after the curl command was run

Once we are inside the psql prompt we can run **\dt** : this command lists all the tables we only have one table authors in our authorsdb database)

then to view all rows we can SELECT * FROM authors;

INSERTING A ROW IN OUR POSTGRESQL TABLE INSIDE DOCKER

```
authorsdb=# INSERT INTO authors (id, name, age) VALUES (2, 'Gustavo Fring', 58);
INSERT 0 1
authorsdb=# SELECT * FROM authors;
 id | age |     name
----+-----+--------------
  1 |  52 | Walter White
  2 |  58 | Gustavo Fring
(2 rows)

authorsdb=#
```

RAM 1.07 GB CPU 0.00% Disk: 1.80 GB used (limit 1006.85 GB)

Now using curl we will see all the records we sent

curl http://localhost:8080/authors

localhost:8080/authors

Pretty-print


```
[
  {
    "id": 1,
    "name": "Walter White",
    "age": 52
  },
  {
    "id": 2,
    "name": "Gustavo Fring",
    "age": 58
  }
]
```


Command Prompt

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\LDIN>curl -X POST http://localhost:8080/authors -H "Content-Type: application/json"
-d "{\"name\":\"Walter White\", \"age\":52}"
{"id":1,"name":"Walter White","age":52}
C:\Users\LDIN>curl http://localhost:8080/authors
[{"id":1,"name":"Walter White","age":52},{"id":2,"name":"Gustavo Fring","age":58}]
C:\Users\LDIN>
```

From both command line and via we are able to see it ,our data show up both in **our browser (localhost:8080)** *and* inside **Docker PostgreSQL via psql**

Exiting Gracefully

1. Stop Spring Boot Application first red square to stop

To quit psql enter and hit \q ,

then exit to get out of docker container

```
authorsdb=# \q
root@b106b009d5ce:/# exit
exit
PS C:\Users\LDIN>
```

STOPPING AND RERUNNING A DOCKER CONTAINER and checking via docker ps

docker start [container-name]

docker stop [container-name]

```
root@b106b009d5ce:/# exit
exit
PS C:\Users\LDIN> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED       STATUS       PORTS                    NAMES
b106b009d5ce   postgres   "docker-entrypoint.s…"   4 hours ago   Up 4 hours   0.0.0.0:5432->5432/tcp   pgdb
PS C:\Users\LDIN> docker stop pgdb
pgdb
PS C:\Users\LDIN> docker ps
CONTAINER ID   IMAGE      COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\Users\LDIN> docker start pgdb
pgdb
PS C:\Users\LDIN> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED       STATUS         PORTS                    NAMES
b106b009d5ce   postgres   "docker-entrypoint.s…"   4 hours ago   Up 3 seconds   0.0.0.0:5432->5432/tcp   pgdb
PS C:\Users\LDIN>
```

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\LDIN> docker ps

CONTAINER ID IMAGE COMMAND CREATED STATUSPORTS NAMES

ade256622ae7 postgres "docker-entrypoint.s…" 38 minutes ago Up 14 minutes 0.0.0.0:5432->5432/tcp psdatabase-db-1

PS C:\Users\LDIN> docker exec -it psdatabase-db-1 bash

```
root@ade256622ae7:/# psql -U postgres
psql (17.5 (Debian 17.5-1.pgdg120+1))
Type "help" for help.

postgres=# \dt
          List of relations
 Schema |  Name   | Type  |  Owner
--------+---------+-------+----------
 public | authors | table | postgres
 public | books   | table | postgres

postgres=# SELECT * FROM authors;
 id | age | name
----+-----+------
(0 rows)

postgres=# SELECT * FROM books;
 isbn | title | author_id
------+-------+-----------
(0 rows)

postgres=# \ds
                List of relations
 Schema |        Name        |   Type   |  Owner
--------+--------------------+----------+----------
 public | author_id_sequence | sequence | postgres
(1 row)
```

postgres=#

```
Terminal

PS C:\Users\LDIN> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS          PORTS                    NAMES
ade256622ae7   postgres   "docker-entrypoint.s…"   38 minutes ago   Up 14 minutes   0.0.0.0:5432->5432/tcp   psdatabase-db-1
PS C:\Users\LDIN> docker exec -it psdatabase-db-1 bash
root@ade256622ae7:/# psql -U postgres
psql (17.5 (Debian 17.5-1.pgdg120+1))
Type "help" for help.

postgres=# \dt
          List of relations
 Schema |  Name   | Type  |  Owner
--------+---------+-------+----------
 public | authors | table | postgres
 public | books   | table | postgres

postgres=# SELECT * FROM authors;
 id | age | name
----+-----+------
(0 rows)

postgres=# SELECT * FROM books;
```

```
Terminal

postgres=# SELECT * FROM authors;
 id | age | name
----+-----+------
(0 rows)

postgres=# SELECT * FROM books;
 isbn | title | author_id
------+-------+-----------
(0 rows)

postgres=# \ds
                List of relations
 Schema |        Name        |   Type   |  Owner
--------+--------------------+----------+----------
 public | author_id_sequence | sequence | postgres
(1 row)

postgres=#
```